3 Requirements

This chapter follows the department's convention: a requirement-gathering narrative, personas, MoSCoW-prioritised functional and non-functional requirement tables, use cases, and user flows. Requirements state **what the user can do and what the system must guarantee** — how any of it is achieved belongs to the Design chapter, not here.

3.1 Problem statement

Capable AI assistants exist, but the ones available to consumers are cloud-hosted: whatever the user asks — their email, their calendar, their files — leaves their machine. Assistants that do run locally exist as developer tools: they require command-line installation, manual model management, and constant configuration, placing them out of reach of the people who would benefit most. And in either form, today's assistants have a fixed repertoire: a request outside it simply fails, or worse, is attempted badly and reported as done.

There is a need for a personal assistant that a non-technical person can install and operate on their own computer, that keeps every piece of their data on that computer, that handles the everyday load of email and calendar, that can safely *extend its own abilities* when asked for something new — and that never takes an irreversible action, or installs code into itself, without the user's knowledge and consent.

3.2 Requirement gathering

Direct stakeholder interviews were not feasible; requirements were derived from four sources:

- **The project brief.** The supervisors' proposal specifies the pillars: a locally-run, privacy-first assistant; autonomous skill generation in response to novel requests; asynchronous access through a familiar messaging channel; and evaluation of latency, task success, and the security of sandboxed execution.
- **Measured behaviour of an existing agent runtime.** A pilot study run for this project (Chapter 5) drove a mature open-source agent framework with local models on consumer hardware. Its failure modes — actions reported done that never happened, uncontrolled tool use, self-written capabilities installed without testing — directly motivate the honesty, consent, and verification requirements below.
- **Community evidence.** Published community guidance on running agent frameworks against local models corroborates the pilot: reliable operation is generally reported only with far larger models than consumer hardware can serve, reinforcing the need for a system designed around small-model limits rather than assuming them away.
- **Scenario analysis.** Personas and scenarios (§3.2.1) were mapped against the everyday tasks a personal assistant is expected to carry: correspondence, scheduling, file chores, and repeated ad-hoc requests.

3.2.1 Personas

Scenario. Maya asks the assistant whether a client has replied about a filing deadline; it answers from her own mailbox, on her own machine, in seconds. She then asks it to rename three hundred scanned receipts by date — something it has never done before. It tells her it doesn't yet

Table 1: Persona: Maya Okafor

Maya Okafor	
Role	Self-employed accountant, 52
Background	Runs her practice from a MacBook; competent with ordinary applications, has never used a terminal. Client correspondence is confidential by professional obligation.
Responsibilities	Managing a high volume of client email, deadlines and appointments; safeguarding client financial data.
Challenges	Maya would benefit most from an assistant, but every capable one she has tried requires sending client data to a third-party service, which she cannot justify professionally. Local alternatives assume technical skill she does not have and does not want to acquire. She needs installation and daily use to feel like any other Mac application, and she needs certainty that nothing she asks ever leaves her machine.

Table 2: Persona: Dev Sharma

Dev Sharma	
Role	MSc student, 24
Background	Technically literate; comfortable trying new software; time-poor.
Responsibilities	Coursework deadlines, supervisor correspondence, part-time work scheduling.
Challenges	Dev's questions are small but constant — "did the department reply about my extension?", "what's on Friday?" — and each one costs a context switch into webmail. He wants them answered conversationally, ideally by voice while doing something else, and wants repeated chores to get faster rather than cost the same effort every time.

Table 3: Persona: Ines Almeida

Ines Almeida	
Role	Software developer, 34
Background	Builds backend systems professionally; security-conscious by habit and by trade.
Responsibilities	Evaluating tools before trusting them with real data.
Challenges	Ines's default stance toward an agent that can act on her machine is distrust. She will not use an assistant that acts invisibly: she wants to see what it is doing while it does it, review what any self-installed ability is permitted to touch, veto anything irreversible, and verify — not be told — that claimed outcomes really happened.

have that ability, describes what it would build and what that ability would be allowed to access, and asks permission. She approves; a minute later the receipts are renamed, and the ability remains for next quarter. Ines, watching the same flow, can open the activity view mid-task, read the new ability's permissions, and delete it afterwards if she chooses.

3.2.2 MoSCoW table

Table 4: Functional requirements (MoSCoW-prioritised)

ID	Functional Requirement	Priority
F1	The application shall allow the user to install and set it up without any command-line interaction	MUST
F2	The application shall allow the user to make requests in natural language through a chat interface	MUST
F3	The application shall allow the user to speak requests aloud and hear spoken replies	MUST
F4	The application shall allow the user to read, search and summarise email from their own account	MUST
F5	The application shall allow the user to draft, reply to and send email from their own account	MUST
F6	The application shall allow the user to ask what is on their calendar	MUST
F7	The application shall not take an irreversible action unless the user's request authorised it or the user explicitly approves it when asked	MUST
F8	The application shall offer to build a new skill when a request is beyond its abilities, and shall proceed only with the user's explicit approval	MUST
F9	The application shall test every skill it builds and shall install only those that pass	MUST
F10	The application shall show the user every ability it has, including what each is permitted to access	MUST
F11	The application shall allow the user to remove any learned ability	MUST
F12	The application shall show the user what it is doing while it works	MUST
F13	The application shall report outcomes from verified results and shall report failures as failures	MUST
F14	The application shall not enter, store or ask for the user's passwords or payment details, and shall hand control to the user at any sign-in	MUST
F15	The application shall allow the user to choose which locally-installed models it uses	SHOULD
F16	The application shall make skills it has built usable by other agent software on the user's machine	SHOULD
F17	The application shall allow the user to view its learned web routines and remove them	SHOULD
F18	The application shall allow the user to review past skill-building attempts, including failed ones	SHOULD

Continued on next page

Table 4: Functional requirements (MoSCoW-prioritised) — continued

ID	Functional Requirement	Priority
F19	The application shall allow the user to view measurements of its own performance	SHOULD
F20	The application shall allow the user to have appointments and events booked into their calendar — either as requested directly, or from details the assistant finds in their email — subject to the same approval rules as any other irreversible action (F7)	SHOULD
F28	The application shall allow a request beyond its own abilities to be handed to other agent software on the user's machine, only with the user's explicit approval and with the handoff's reduced guarantees stated	SHOULD
F21	The application shall allow the user to interact with it from their phone	COULD
F22	The application shall allow the user to make quick requests without opening the main window	COULD
F23	The application shall allow the user to browse their past conversations	COULD
F24	The application shall allow the user to share a skill it has built with another installation	COULD
F25	The application shall automate websites beyond the user's mail and calendar	WON'T
F26	The application shall run on platforms other than Apple-silicon macOS	WON'T
F27	The application shall support multiple users on one installation	WON'T

Table 5: Non-functional requirements (MoSCoW-prioritised)

ID	Non-Functional Requirement	Priority
NF1	The application shall keep the user's data, and all processing of it, on the user's machine, and shall not transmit it to any third-party service	MUST
NF2	The application shall not allow content it encounters while working — a webpage, an email, a document — to authorise actions	MUST
NF3	The application shall be usable by a first-time, non-technical user without documentation	MUST
NF4	The application shall operate fully on a consumer machine with 24 GB of unified memory	MUST
NF5	The application shall acknowledge every request within a few seconds, answer routine questions in under a minute, and complete typical mail tasks within about two	MUST

Continued on next page

Table 5: Non-functional requirements (MoSCoW-prioritised) — continued

ID	Non-Functional Requirement	Priority
NF6	The application shall recover from interruption gracefully, and shall never present a failed or unfinished action as a completed one	SHOULD
NF7	The application shall become faster at tasks it has performed before	SHOULD
NF8	The application shall be accompanied by documented code, a system manual and a user manual	SHOULD

Scope remarks. F25 is excluded on evidence rather than preference: the pilot measured the baseline runtime at 1/7 on general web tasks with local models, and community guidance places reliable agentic browsing at model sizes beyond consumer hardware; mail and calendar (F4–F6) remain in scope because the pilot verified them end-to-end. F21 is held at COULD as an inexpensive extension of the same request pipeline over a messaging transport. The prohibitions are deliberately MUSTs rather than WON'Ts: F7, F14 and NF1 are binding properties of every release, not features deferred from this one.

3.3 Use cases

Table 6: Use-case summary

ID	Actor	Goal	Requirements
UC1	Maya	Install the assistant and reach a first successful answer	F1, F2, NF3
UC2	Dev	Ask, by voice, whether someone has replied	F3, F4, F13
UC3	Dev	Send a reply by voice	F3, F5, F7, F13
UC4	Maya	Ask for something new; approve the skill the assistant offers to build	F8, F9, F10
UC5	Dev	Repeat a previously learned task and see it complete faster	F10, NF7
UC6	Ines	Inspect an ability's permissions; remove it	F10, F11, F18
UC7	Ines	Watch the assistant work; review its performance measurements	F12, F19
UC8	Maya	Reach a login wall and take over herself	F14
UC9	Dev	Ask from his phone	F21
UC10	Maya	Have an appointment from an email put on her calendar	F20, F7, F13

3.3.1 UC1 — First run to first answer

- *Precondition:* installer obtained; clean machine.
- *Trigger:* first launch.

1. The assistant checks the machine meets its needs and says what it will download and why.
2. The user accepts the defaults; downloads proceed with visible progress and survive interruption.

3. Everything needed starts without further action; the assistant invites a first request.
4. The user asks a question and receives an answer.

- *Error paths:* machine below requirements → the missing requirement is named and setup stops cleanly; interrupted download → resumes on next launch.

3.3.2 UC3 — Send a reply by voice

- *Precondition:* the user has connected their mail account (F14: by signing in themselves).
- *Trigger:* the user says "reply to `<address>` saying I'll be there at nine."

1. The transcript is shown; the activity view narrates the steps (F12).
2. Because the spoken request itself asks for a reply, sending is authorised without a second prompt (F7).
3. The outcome is confirmed from the mailbox itself, and spoken back (F13).

- *Variant:* "draft…" → composed but never sent, and the user is told where it is.
- *Error path:* the named person is ambiguous in this mailbox → the assistant declines to guess, lists the candidates, and sends nothing.

3.3.3 UC4 — Approve a new skill

- *Trigger:* "rename all the receipts in this folder by their dates."

1. No installed ability covers it; the assistant offers to build one, stating what it would do, what it would be allowed to access, and roughly how long it will take (F8).
2. The user approves; progress and test results stream in the activity view (F12).
3. Tests pass → the skill is installed, run, and the result presented; it now appears among the user's abilities with its origin recorded (F9, F10).
4. Had the user declined, nothing would have been created, and the offer lapses on its own (F8).

- *Error path:* attempts exhaust without passing tests → an honest failure with the reason; nothing is installed (F9, F13).

3.4 User flows

3.5 Verification mapping (feeds Chapter 5)

3.6 Decisions log

- 2026-08-09 — Phone access (F21) held at COULD.
- 2026-08-09 — General web automation excluded (F25) on measured and community evidence, recorded in §3.2.2's scope remarks.
- 2026-08-09 — Requirements rewritten solution-agnostically; all implementation nouns moved out of this chapter.

Flow diagram — exported from architecture sources

1. A user request arrives, typed or spoken, and is matched against the assistant's understood abilities.
2. A question → answered from local knowledge; the reply is shown or spoken.
3. An installed ability → it is run; the reply is shown or spoken.
4. Mail or calendar → the assistant works the account. At an irreversible step: if the request authorised it, the assistant does it and verifies the outcome; if not, it asks the user — approval leads to the same verified action, decline ends with a reply and nothing done.
5. Nothing covers it → the assistant offers to build a skill. If approved, it builds and tests: tests pass → install and run, then reply; tests fail → an honest failure report. If declined or lapsed, the interaction ends with a reply and nothing created.

Figure 1: Request flow (every interaction takes this path)

Flow diagram — exported from architecture sources

1. First launch.
2. Machine check — if the machine is below requirements, the missing requirement is named and setup stops cleanly.
3. Confirm downloads.
4. Downloads proceed with visible progress and are resumable.
5. Everything starts.
6. The user connects mail, signing in themselves.
7. First request.

Figure 2: First-run flow

Flow diagram — exported from architecture sources

1. Hold to talk.
2. On-device transcription.
3. Transcript shown.
4. Request flow (Figure 1).
5. On-device speech.
6. Spoken reply.

Figure 3: Voice flow

Table 7: Verification mapping

Requirement(s)	Verified by
F4–F7, F13, NF5	Task suite A of the evaluation protocol, outcomes checked against the mailbox itself
F20	A booking case added to suites A and D, outcome checked against the calendar itself
F8, F9	Consent tests (automated); skill-generation suite C with damage rate
F7, F14, NF2	Safety suite D, including the adversarial page scenario
NF7, F17	Repeat-cost suite E
F1, NF3	Scripted clean-machine installation; UAT ($n \geq 4$) with SUS
NF1	Network egress log captured across a full demo script
NF4	Existing memory-residency measurements

- 2026-08-10 — Requirements restated in the department's convention: every row "the application shall / shall not", prohibitions promoted from WON'T to binding MUSTs, non-functional categories removed.
- 2026-08-10 — Calendar booking added (F20, SHOULD): events booked on request or from details found in email, behind the F7 approval rules; scheduled at the top of Phase 2 with a gated week-4 stretch slot.